

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1

по дисциплине «Проектирование защищенных интеллектуальных
информационных систем»

на тему «Управление доступом с помощью списков контроля доступа.
Часть 2. Управление доступом к приложениям и системам баз данных»

Выполнил студент гр. 321701

Ковальчук В. Н.
Самович В. М.

Проверил

Сальников Д. А.

Минск 2026

Цель: изучить механизмы контроля доступа, предоставляемые серверными приложениями и системами управления базами данных, и получить практические навыки настройки политики безопасности (администратор – пользователь – гость) для СУБД и веб-приложений.

Задание:

- а) Создать программу, которая позволяет установить веб-приложение или систему управления базами данных (СУБД).
- б) Создать программу, которая позволяет настроить механизм контроля доступа для выбранного веб-приложения или СУБД.
- в) Создать программу для проверки корректности работы настроенной политики безопасности выбранной СУБД или веб-приложения.
- г) Создать программу, которая позволяет удалить веб-приложение или СУБД, а также созданные ими объекты файловой системы.
- д) Оформить отчёт о проделанной работе.

Политика безопасности должна быть настроена следующим образом: в системе должны присутствовать администратор системы с полным доступом ко всем подсистемам и максимальными привилегиями, пользователь с полными либо частичными правами в подсистеме, предоставленной администратором, и гость с правами только на чтение отдельных фрагментов, предоставленных администратором.

Задание выполнено для документоориентированной СУБД MongoDB. В виде отдельного консольного C#-проекта (Lab1.2-MongoDB) реализован сценарий, последовательно выполняющий все четыре программы из задания как этапы одного сценария: установка, настройка контроля доступа, проверка и удаление. Все действия – реальные команды `bash` (`apt-get`, `mongod`, `mongosh`, `npm` и т. п.), запускаемые как внешние процессы; .NET здесь выступает только оркестратором сценария, а не исполнителем самой политики безопасности.

Краткое описание MongoDB

MongoDB – документоориентированная СУБД: данные хранятся не в таблицах со строками, а в коллекциях JSON-подобных документов (BSON). В лабораторной работе сервер устанавливается напрямую на хост из официального apt-репозитория пакетом `mongodb-org` – без Docker, теми же средствами `bash`, что и в первой части лабораторной работы. По умолчанию свежий экземпляр MongoDB принимает любые подключения без аутентификации – контроль доступа необходимо включить явно ключом `-auth` и настроить отдельно от установки. Модель прав в MongoDB строится вокруг ролей: встроенные роли (например, `root`, `readWrite`) назначаются

пользователю в контексте конкретной базы данных, а для более точных ограничений можно определить собственную роль с перечнем разрешённых действий над конкретной коллекцией – этим MongoDB напоминает списки контроля доступа из первой части лабораторной работы, только уровнем выше: не файл и не каталог, а документ и коллекция.

Реализация: MongoDB

Проект `Lab1.2-MongoDB` организован так же, как и программа из первой части лабораторной работы: `Program.cs` последовательно вызывает четыре этапа из каталога `Steps` (`Install`, `Configure`, `Verify`, `Cleanup`), а вспомогательный класс `Report` запускает внешние процессы и печатает результат каждой команды в формате `[OK]/[FAIL]`, а результат проверки прав доступа – в формате `[ALLOWED]/[DENIED]`.

1. Установка

Установка выполняется командой `sudo apt-get install -y mongodb-org`, после чего создаётся каталог для постоянного хранения данных (`/var/lib/mongo_lab_data`) и сервер запускается напрямую как обычный процесс: `sudo mongod -fork -dbpath ... -logpath ....`. Ключ `-fork` здесь играет ту же роль, что и `-d` у `Docker`: команда демонизирует процесс и сразу возвращает управление сценарию. Сервер намеренно запускается без `-auth`: MongoDB разрешает создать самого первого пользователя без пароля только через т. н. `localhost exception`, то есть до включения обязательной аутентификации.

2. Настройка контроля доступа

Пока аутентификация ещё не включена, программа создаёт администратора `admin` с встроенной ролью `root`, после чего процесс `mongod` останавливается (`mongod -shutdown`) и запускается заново с ключом `-auth` на том же каталоге данных, поэтому созданный пользователь сохраняется. Далее от имени администратора создаются: пользователь `labuser` с ролью `readWrite` на базе `labdb` (полные права в предоставленной администратором «подсистеме» – одной прикладной базе данных) и пользователь `labguest`, которому назначена не встроенная, а специально созданная роль `guestReader`, разрешающая только действие `find` и только над одной коллекцией `public_notes` (см. листинг 1). Соседняя коллекция `private_notes` для гостя недоступна вовсе.

```

1 Report.Step(8, $"Create '{Config.GuestUser}' - read-only on
  ↳ '{Config.AppDb}.{Config.PublicCollection}' only");
2 MongoEvalAuth($"Create role '{Config.GuestRole}' (find on
  ↳ {Config.AppDb}.{Config.PublicCollection} only)",
3   $"db.getSiblingDB('{Config.AppDb}').createRole({{role:'{Config.GuestRol}
  ↳ e}','",
  ↳ +
4   $"privileges:[{{resource:{{db:'{Config.AppDb}',collection:'{Config.Publ}
  ↳ icCollection}}}},actions:['find']}}],roles:[]}})");
5 MongoEvalAuth($"Create user '{Config.GuestUser}' (role {Config.GuestRole})",
6   $"db.getSiblingDB('{Config.AppDb}').createUser({{user:'{Config.GuestUse}
  ↳ r}','{Config.GuestPassword}',roles:[{{role:'{Config.GuestRole}'
  ↳ ,db:'{Config.AppDb}'}}]}})");

```

Листинг 1 – Создание гостевой роли и пользователя с правом чтения одной коллекции

Для визуальной проверки той же настройки устанавливается веб-интерфейс Mongo Express – пакетом `npm install -g mongo-express`, без контейнера. Поскольку сам процесс не демонизируется самостоятельно (в отличие от `mongod -fork`), сценарий запускает его в фоне средствами `.NET` (`Process.Start` без ожидания завершения) – прямой аналог `docker run -d` для обычного процесса.

3. Проверка корректности

Проверка выполняется той же утилитой `mongosh`, но уже от имени каждой из трёх учётных записей, и интерпретирует нулевой код завершения команды как **ALLOWED**, а ненулевой (сервер отклонил операцию как неавторизованную) – как **DENIED**. Полный протокол реального запуска программы приведён ниже (см. листинг 2). Администратор проходит любую проверку; **labuser** может читать и писать в **labdb**, но не может ни создать нового пользователя, ни обратиться к посторонней базе **otherdb**; **labguest** может прочитать только **public_notes** – чтение **private_notes** и любая запись отклоняются. Отдельно проверено, что клиент без учётных данных не может выполнить вообще ни одной операции – **-auth** действительно принудительно применяется сервером, а не является лишь опцией по умолчанию.

```

1  == Step 1: Install the MongoDB server package ==
2  [OK] Install mongodb-org
3  $ sudo apt-get install -y mongodb-org
4  == Step 2: Create a persistent data directory ==
5  [OK] Create data directory
6  $ sudo mkdir -p /var/lib/mongo_lab_data
7  == Step 3: Start MongoDB (bootstrap mode, no authentication yet) ==
8  [OK] Start mongod
9  $ sudo mongod --fork --dbpath /var/lib/mongo_lab_data --port 27017 --logpath /var/log/mongo_lab.log --bind_ip_all
10 [OK] MongoDB accepted a connection and is ready
11 == Step 4: Create the administrator account through the localhost exception ==
12 [OK] Create admin user (role 'root' on 'admin')
13 $ mongosh --port 27017 --quiet --eval db.getSiblingDB('admin').createUser({user:'admin',pwd:'AdminP@ss1',roles:[{role:'root',db:'admin'}]})
14 == Step 5: Restart MongoDB with authorization enforced ==
15 [OK] Stop the bootstrap mongod
16 $ sudo mongod --shutdown --dbpath /var/lib/mongo_lab_data
17 [OK] Start mongod with --auth
18 $ sudo mongod --fork --dbpath /var/lib/mongo_lab_data --port 27017 --logpath /var/log/mongo_lab.log --bind_ip_all --auth
19 [OK] MongoDB is back up with authorization enforced
20 == Step 6: Seed the application database with public and private data ==
21 [OK] Insert a document into public_notes
22 $ mongosh --port 27017 -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
23   ↪ db.getSiblingDB('labdb').public_notes.insertOne({title:'public seed', visibility:'public'})
24 [OK] Insert a document into private_notes
25 $ mongosh --port 27017 -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
26   ↪ db.getSiblingDB('labdb').private_notes.insertOne({title:'private seed', visibility:'private'})
27 == Step 7: Create 'labuser' - readWrite on 'labdb' only ==
28 [OK] Create user 'labuser' (role readWrite on labdb)
29 $ mongosh --port 27017 -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
30   ↪ db.getSiblingDB('labdb').createUser({user:'labuser',pwd:'UserP@ss1',roles:[{role:'readWrite',db:'labdb'}]})
31 == Step 8: Create 'labguest' - read-only on 'labdb.public_notes' only ==
32 [OK] Create role 'guestReader' (find on labdb.public_notes only)
33 $ mongosh --port 27017 -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval db.getSiblingDB('labdb').createRole({role:'guest
34   ↪ Reader',privileges:[{resource:{db:'labdb',collection:'public_notes'},actions:['find']}],roles:[]})
35 [OK] Create user 'labguest' (role guestReader)
36 $ mongosh --port 27017 -u admin -p AdminP@ss1 --authenticationDatabase admin --quiet --eval
37   ↪ db.getSiblingDB('labdb').createUser({user:'labguest',pwd:'GuestP@ss1',roles:[{role:'guestReader',db:'labdb'}]})
38 == Step 9: Start Mongo Express so the setup can be checked visually ==
39 [OK] Install Mongo Express
40 $ sudo npm install -g mongo-express@1.0.0
41 [OK] Start Mongo Express
42 $ mongo-express &
43 [OK] Open http://localhost:8081 to browse 'labdb' (public_notes/private_notes) visually, connected as 'admin'
44 == Step 10: Verify the administrator's privileges ==
45 [ALLOWED] admin ListDatabases admin (cluster-wide)
46 == Step 11: Verify 'labuser' has full rights inside 'labdb' only ==
47 [ALLOWED] labuser Read labdb.public_notes
48 [ALLOWED] labuser Write labdb.private_notes
49 [DENIED ] labuser CreateUser labdb (admin-only op)
50 [DENIED ] labuser Read otherdb.secrets (no role granted)
51 == Step 12: Verify 'labguest' can only read 'public_notes' ==
52 [ALLOWED] labguest Read labdb.public_notes
53 [DENIED ] labguest Read labdb.private_notes
54 [DENIED ] labguest Write labdb.public_notes
55 == Step 13: Verify that an unauthenticated client is rejected outright ==
56 [DENIED ] (none) Write labdb.public_notes (no credentials)
57 == Step 14: Stop MongoDB, Mongo Express and remove the data directory and packages ==
58 [OK] Stop mongod
59 $ sudo mongod --shutdown --dbpath /var/lib/mongo_lab_data
60 [OK] Stop Mongo Express
61 $ sudo pkill -f mongo-express
62 === Summary ===
63 Operations: 18 total, 0 failed
64 CreateUser allowed 0 / 1 denied 1
65 ListDatabases allowed 1 / 1 denied 0
66 Read allowed 2 / 4 denied 2
67 Write allowed 1 / 3 denied 2

```

Листинг 2 – Реальный протокол работы программы Lab1.2-MongoDB

4. Удаление

Завершающий этап останавливает процессы `mongod` и `Mongo Express` (`mongod -shutdown` и `pkill -f mongo-express`). Удаление каталога с данными и самих пакетов (`mongodb-org` и глобального `npm`-пакета `mongo-express`) намеренно закомментировано в коде – команды присутствуют, но не выполняются по умолчанию, – чтобы при повторных запусках демонстрации не приходилось заново скачивать и устанавливать пакеты, а данные не терялись без явного намерения; при необходимости полной очистки соответствующие строки достаточно раскомментировать.

Выводы

В ходе выполнения лабораторной работы был освоен механизм контроля доступа на уровне приложений и систем управления базами данных – в отличие от первой части, где права ограничивались правами файловой системы, здесь для MongoDB реализована и экспериментально подтверждена трёхуровневая политика безопасности «администратор – пользователь – гость», выраженная собственными, специфичными для этой СУБД средствами: встроенными и пользовательскими ролями с привилегиями на уровне базы данных и коллекции. Практически показано, что в основе этого механизма лежит принцип наименьших привилегий: пользователю и гостю явно и по отдельности выдаётся ровно тот объём прав, который необходим для их роли, а не урезается из более широкого набора.

Семантически лабораторная работа подтвердила, что задача администратора не сводится к однократной установке ПО: то, что MongoDB по умолчанию разрешает подключение вовсе без аутентификации, означает, что первым шагом всегда является явное включение и настройка контроля доступа – то же наблюдение, что и в первой части работы, только теперь на уровне не операционной системы, а конкретного приложения. Установка средствами `bash` напрямую на хост (пакет `mongodb-org` и демонизация процесса через `mongod -fork`, без Docker) также оказалась практичным решением: она позволила выполнять реальные, а не имитационные команды установки и настройки теми же средствами, что и в первой части лабораторной работы, и гарантировала воспроизводимую остановку всех созданных процессов на завершающем этапе.